Voice-Based Fault Reporting Device

End-to-end IoT platform — firmware, backend and dashboard.
Project report and operating manual.

Status	Working end to end. Deployed and in bench testing.
Live URL	https://voice-complaint-system.vercel.app
Repository	github.com/Srinivas0694/voice-complaint-system
Hardware	ESP32-S3-WROOM-1 N16R8 + INMP441 I2S microphone
Recurring cost	Rs 0 / month — every service on a free tier
Report date	23 July 2026

1. What it does

A compact device is fixed to an appliance — a freezer, air conditioner or washing machine. When something goes wrong the user holds a button and speaks the fault aloud. Releasing the button uploads the recording. Within a few seconds the complaint appears on a web dashboard: transcribed into text, classified by fault type and severity, and tagged with the installation location.

No app to install, no form to fill, no typing. The person who notices a problem is often not the person who files reports, and speaking is the lowest-friction way to capture what they saw.

The complete path

```
button held      →  ESP32-S3 captures 16 kHz mono audio over I2S
                 →  buffered in PSRAM (480 KB for 15 seconds)

button released  →  HMAC-signed multipart HTTPS upload
                 →  Vercel receives it
                     →  WAV stored in Supabase Storage
                     →  complaint row written, status = PROCESSING
                     →  Groq Whisper transcribes the audio
                     →  Groq Llama classifies appliance / fault / severity
                     →  row updated, status = OPEN
                     →  device location attached from its record

                 →  dashboard shows transcript, waveform and severity
```

Typical end-to-end time is 3 to 5 seconds. If the network is unavailable the complaint is written to flash and uploads itself when the connection returns — carrying its original timestamp, not the reconnection time.

2. What was built

Component	Detail	Size
Firmware	PlatformIO project for ESP32-S3. FreeRTOS tasks, state machine, durable offline queue, watchdog, OTA-ready.	1,953 lines 20 files
Backend	Next.js 15 Route Handlers covering ingest, authentication, complaints, devices and dashboard statistics.	16 routes
Dashboard	Next.js App Router with Tailwind, Framer Motion and Recharts. Login, overview, complaint list, detail, devices.	5 pages
Database	PostgreSQL through Prisma — users, devices, complaints, events, heartbeats, firmware releases, refresh tokens, nonces.	8 tables
Scripts	Device registration, admin seeding, password reset, classification backfill, hardware-free ingest test.	5 scripts
Documentation	Architecture, hardware design with BOM, free-tier analysis, setup guide, full REST API contract.	5 documents

Backend and dashboard together come to 3,216 lines of TypeScript. The repository holds 91 files.

3. Technology choices and why

Layer	Chosen	Reasoning
MCU	ESP32-S3 N16R8	8 MB PSRAM holds the whole recording in RAM, so flash is written only when an upload fails. 16 MB flash gives comfortable dual-OTA slots.
Microphone	INMP441 I2S MEMS	Digital output, no analog noise floor.
Transport	Wi-Fi + HTTPS	2G GPRS cannot move 480 KB of audio in acceptable time, and the A9G module's TLS stack is not dependable.
Speech to text	Groq whisper-large-v3-turbo	Free tier with no billing account. Handles Indian-accented English well.
Classification	Groq llama-3.3-70b	A second free call on the transcript returns appliance, fault category and severity.
Backend	Next.js Route Handlers	Transcription returns inside the serverless limit, so there is no queue, no worker and no second deploy.
Database	Supabase PostgreSQL	Relational data with real foreign keys. Free tier needs no card, and storage lives in the same project.
Location	Provisioned per device	An appliance monitor never moves. An installer dropping a pin beats a GNSS fix in a basement kitchen.
Device auth	HMAC-SHA256 signing	JWT rotation on a constrained device is misery. A per-device secret in encrypted NVS is simpler and stronger.

4. Running the system

Everything below runs from PowerShell on Windows. Two terminal windows are used: one holds the server and is then left alone, the other runs everything else.

Paths

```
Project    C:\Users\srini\Downloads\vcs-v2\VoiceComplaintSystem
Backend    ...\VoiceComplaintSystem\Frontend
Firmware   ...\VoiceComplaintSystem\Firmware
```

4.1 Start the backend locally

```
cd C:\Users\srini\Downloads\vcs-v2\VoiceComplaintSystem\Frontend

npm install                    # first time only
npx prisma generate            # after any schema change

npm run dev                    # development — hot reload, slow first paint

# or, for a production build:
npm run build
npm start
```

Leave that window running and open a second terminal for everything else. Use **npm run build** before deploying — it type-checks properly, which **npm run dev** does not.

4.2 Check it is alive

```
curl.exe http://localhost:3000/api/health
curl.exe https://voice-complaint-system.vercel.app/api/health

# expected:
#   {"status":"ok","db":"up","latencyMs":169,...}
```

4.3 Register a device

```
npx tsx scripts/register-device.ts DEVICE_001 "Bench prototype" FREEZER \
    --lat 17.4126 --lng 78.4482 --site "Sentiantal, Hyderabad"

# rotate the secret only, keeping everything else:
npx tsx scripts/register-device.ts DEVICE_001 --rotate
```

This prints a 64-character device secret **once**. It is encrypted at rest and cannot be read back. Copy it straight into secrets.ini.

4.4 Test the whole pipeline with no hardware

```
$env:TEST_DEVICE_ID     = "DEVICE_001"
$env:TEST_DEVICE_SECRET = "<the 64-hex secret>"
$env:API_BASE_URL       = "https://voice-complaint-system.vercel.app/api/v1"

npx tsx scripts/test-ingest.ts .\sample.wav

# expected:
#   {
#     "status":        "OPEN",
#     "transcript":    "The freezer is not cooling.",
#     "applianceType": "FREEZER",
#     "faultCategory": "COOLING",
#     "severity":      "CRITICAL"
#   }
```

This synthesises a device — it builds the request, signs it exactly as the ESP32 does, and posts it. Run it before touching hardware. A verified backend turns hardware debugging from a two-variable problem into a one-variable problem.

4.5 Flash the firmware

```
cd ..\Firmware

pio run -e prototype -t upload      # build and flash
pio device monitor --port COM8      # watch the logs

pio run -e prototype -t clean       # after changing headers
pio run -e prototype -t erase       # wipe flash, clearing the outbox
```

If upload fails with "no serial data received": hold BOOT, tap RESET, release BOOT, then upload again. Use the port labelled **USB**, not UART — the firmware logs over native USB CDC.

4.6 Administration and maintenance

```
npx tsx scripts/seed.ts                       # create the first admin
npx tsx scripts/set-password.ts admin@example.com "new password"

npx tsx scripts/backfill-classify.ts          # dry run, writes nothing
npx tsx scripts/backfill-classify.ts --apply  # write

npx prisma db push                            # apply schema changes
npm run db:studio                             # browse the database
```

4.7 Deploy

```
cd C:\Users\srini\Downloads\vcs-v2\VoiceComplaintSystem

git add .
git commit -m "describe the change"
git push

# Vercel builds automatically — watch Deployments for "Ready".
```

The Vercel root directory must be set to **Frontend**, or the build fails looking for package.json at the repository root.

5. Hardware

Wiring — prototype

ESP32-S3	INMP441	Note
GPIO4	SD	I2S data in
GPIO5	SCK	bit clock
GPIO6	WS	word select
3V3	VDD	3.3 V part, not 5 V
GND	GND and L/R	L/R to ground is mandatory
GPIO0	—	BOOT button, used as the record trigger
GPIO48	—	onboard WS2812 status LED

Two settings that cost a full afternoon to find

Channel format must be ONLY_LEFT. With L/R tied to ground the INMP441 transmits on the left slot. A classic ESP32 needs ONLY_RIGHT for identical wiring because its I2S peripheral swaps the channels; the S3 follows the specification. Getting this wrong reads exactly like a dead microphone — every sample returns 0 or 1.

The microphone carries a large DC offset. Idle samples sit near -30,000,000 rather than zero. Without subtracting a running mean before the gain shift, every sample clips and the transcript comes back as noise.

Pins that must not be used on the N16R8: GPIO26–32 (SPI flash) and **GPIO33–37 (octal PSRAM)**. The PSRAM pins are broken out on the module and look free. They are not.

Status LED

Colour	Meaning
White pulse	booting
Green breath	idle, connected
Amber slow blink	idle, no network
Solid blue	recording
Cyan fast blink	uploading
Solid green, 2 s	delivered and transcribed
Amber double-blink	queued offline, will retry
Solid red, 3 s	rejected or failed

6. Problems solved during the build

Recorded because most cost real time, and would cost it again.

Symptom	Cause and fix
~40 unrelated compiler errors	A local src/wifi.h shadowed the framework WiFi.h on a case-insensitive filesystem. Renamed to netlink.h.
HEX macro redefinition	Arduino's Print.h defines HEX as 16. The local constant became HEX_CHARS.
esp_task_wdt_config_t undeclared	Arduino core 2.0.x is ESP-IDF 4.4, which has the two-argument esp_task_wdt_init.
File system is not mounted	LittleFS defaults to a partition labelled "spiffs"; ours is "littlefs" and must be named explicitly.
Microphone reads 0 or 1 always	Wrong I2S channel — ONLY_RIGHT became ONLY_LEFT. Diagnosed by tying GPIO4 to 3V3 then GND and watching raw values.
SSL invalid record on upload	The firmware always used TLS; the local server is plain HTTP. Transport now follows the URL scheme.
Gemini 429 with limit: 0	Google's free tier now requires a linked billing account. Groq became the primary provider.
Vercel deployment blocked	The git commit author email had a typo and did not match the GitHub account.
git push returned HTTP 408	A committed WAV made the push large enough to time out on a poor connection. Removed and gitignored.
Dashboard spinner never resolved	The API uses two response envelopes; the client assumed one, so stats read undefined without ever erroring.
Build fails on a route export	A Next.js route file may export only HTTP method handlers. The shared filter moved into lib/.

7. Cost

Service	Free allowance	Used for
Vercel Hobby	100 GB bandwidth, 1 M invocations per month	dashboard and API
Supabase Free	500 MB Postgres, 1 GB storage	database and audio files
Groq	generous per-day limits	transcription and classification
GitHub	unlimited private repositories	source control

Two limits worth planning for

Vercel Hobby forbids commercial use. Acceptable for a pilot or a demonstration. Charging a customer requires the Pro plan, around $20 per month, or self-hosting.

Storage caps at roughly 2,000 complaints at 480 KB each. Deleting audio older than 90 days makes this effectively unbounded — the transcript stays in the database regardless.

An Oracle Cloud Always Free ARM instance (4 cores, 24 GB RAM) removes both constraints and can self-host Whisper, at the cost of owning the operations.

8. What remains

Item	Note
Battery and charging	Deferred until after the prototype. An 18650 cell, TP4056 charger and TPS63020 buck-boost — a plain buck browns out the S3 on a sagging cell.
Custom PCB	Board design, BOM and enclosure notes are in the Hardware documentation. About Rs 2,000 per unit at prototype quantity.
Map on the detail page	Coordinates are stored and exported; the detail page does not yet render a map.
Excel and PDF export	CSV works. The other two formats are stubbed.
Transcript accuracy	Short utterances are occasionally misheard. Microphone placement and speaking pace matter more here than any code change.
2G fallback	The A9G driver remains in the tree behind ENABLE_GPS. Useful for sites without Wi-Fi, though a SIM7670G would be the better module.

9. Repository layout

```
VoiceComplaintSystem/
    Firmware/               PlatformIO project (ESP32-S3)
        src/                20 source files
        tools/              A9G serial bridge — a bench diagnostic
        platformio.ini      build config, secrets injection
        partitions.csv      dual OTA slots + LittleFS outbox

    Frontend/               Next.js app — dashboard AND API
        src/app/api/v1/     16 route handlers
        src/app/(app)/      dashboard pages
        src/lib/            auth, crypto, storage, STT providers
        scripts/            registration, seeding, testing, backfill
        prisma/             database schema

    Database/               schema and ER diagram
    Documentation/          architecture, free-tier analysis, setup guide
    Hardware/               pin map, power design, BOM, PCB notes
    API Docs/               full REST contract
```

Two files are deliberately excluded from the repository and must be kept safe separately: **Frontend/.env** and **Firmware/secrets.ini**. Losing DEVICE_SECRET_ENC_KEY means every device in the field needs re-flashing, because device secrets can no longer be decrypted.